LeRobot (Python, English names) User Manual

10 November 2026

LeRobot is a drawing robot you steer on screen: either live, with the mouse and keyboard (Draw), or by writing a short Python program of robot commands (such as `my_drawing.py`). This is the edition with English command names, used by the book *First Programming Lessons in Python*: the robot moves forward with `fd()`, turns a quarter turn to the right with `qtr()`, puts its pen down with `pd()`, and so on. It is the same robot as the French edition (folder `LeRobot_Python_FR`, commands `av()`, `pqtd()`, `bc()`, ...), itself translated from the OCaml and 1998 Caml Light versions: same window, same drawings. It lives in the folder `LeRobot_Python_EN` of the `LeRobot` folder, which can be anywhere (commands below are typed in a Terminal window opened in the `LeRobot` folder); the installation is described in the *LeRobot (Python, English names) Installation Manual* (`LeRobot_Install.pdf`).

Contents

1 Getting started **1**

2 Part 1 — Interactive drawing (Draw) **2**
- 2.1 Launch . . . 2
- 2.2 The window . . . 2
- 2.3 Moving the robot with the mouse or the arrow keys . . . 2
- 2.4 Keyboard commands . . . 2
- 2.5 Finishing and saving . . . 3

3 Part 2 — Running a robot program **3**
- 3.1 Run my_drawing.py . . . 3
- 3.2 Write your own program . . . 4
- 3.3 Programs of the book . . . 4
- 3.4 Replaying at another speed . . . 4
- 3.5 Command reference . . . 5

4 Troubleshooting **6**

1 Getting started

You need a Python linked to Tk 8.6 or later, for example the one from python.org (see the installation manual). There is nothing to compile and no XQuartz: the robot window is an ordinary macOS window.

There are three ways to start a program:

- **The application** `LeRobotPythonEN.app` (in `LeRobot_Python_EN`): double-click it to run Draw; drop a `.py` file on its icon to run that program.

- **The script** `robot.sh`, in a Terminal window opened in `LeRobot_Python_EN`: `./robot.sh path/to/Program.py`. The program can be in any folder.
- **Python directly**, for the files of the `LeRobot_Python_EN` folder itself: `python3 Draw.py`, `python3 my_drawing.py`.

If the `python3` of the Terminal is the one of macOS (linked to the old Tk 8.5), LeRobot notices it and starts itself again with the python.org Python; nothing has to be done.

The window is 800 × 700 points. The large white area is where the robot draws; the strip along the bottom is the message bar.

2 Part 1 — Interactive drawing (Draw)

In Draw you give the robot commands with single keys and move it with the mouse or the arrow keys; press Return when you are done, then `y` to save the drawing as a program.

2.1 Launch

Double-click `LeRobotPythonEN.app`, or:

```
cd LeRobot_Python_EN && python3 Draw.py
```

The robot appears in the centre, pointing north. The window comes to the front with the keyboard focus; if keys do nothing, click in the window.

2.2 The window

- **Drawing area** — the large white area.
- **Message bar** (bottom) — a reminder of the keys, ending with `* :pc w :paint  Return :end`.
- **Colour box** (small robot near the right of the bar) — shows the current pen colour.
- **Return icon** (far right) — the Return symbol; it blinks when the robot waits for a key.

2.3 Moving the robot with the mouse or the arrow keys

Click on a grid point (or move the pointer with the button held down) and the robot jumps to the nearest grid point, without drawing. The arrow keys move it one grid step left, right, up or down, also without drawing. When the pointer moves with the button released, the robot does not move.

If the robot would go past the edge of the drawing area, it stays where it is and the terminal prints `** EXIT FORBIDDEN ** translate`, with a beep-beep. This is only a warning.

2.4 Keyboard commands

The robot draws only when its pen is down (`d`; it is down at the start). One step is one grid square.

Key	Command	What it does
`d`	pd	Pen down: the robot draws as it moves
`u`	pu	Pen up: the robot moves without drawing
`f`	fd	Move forward one step
`)`	cr	Curve to the right (a quarter circle)
`(`	cl	Curve to the left (a quarter circle)
`+`	qtr	Quarter turn right (90°)
`-`	qtl	Quarter turn left (90°)
`r`	etr	Eighth turn right (45°)
`l`	etl	Eighth turn left (45°)
`c`	dc	Draw a small cross (+)
`p`	dp	Draw a point
`.`	dg	Draw the grid points
`*`	pc	Next pen colour (shown in the colour box)
`w`	paint	Paint the closed area behind the robot with the pen colour
Arrow keys	translate	Move one step left, right, up or down, without drawing
Backspace	—	Undo the last command
Return	—	Finish the drawing

Colours cycle in this order: white, yellow, blue, violet, green, red, indigo, black. To paint, put the robot so that its back is inside a closed area, then press `w`.

2.5 Finishing and saving

1. Press **Return**. The bar asks: *Do you want to save in my_drawing.py? (y/n)*
2. Press `y` to save. Every command you used, including mouse and arrow moves (written as `translate(dx, dy)`) and colour changes (written as `pc(n)` with the colour name in a comment), is written to `my_drawing.py` as a Python program (see Part 2), and the window closes.
3. Press any other key to close without saving.

Saving always overwrites `my_drawing.py`, in the folder of Draw. To keep an earlier drawing, copy that file under another name first.

3 Part 2 — Running a robot program

A robot program is a Python file that lists robot commands in order. It draws the picture by itself, then waits so you can replay it at another speed or close it with Esc.

3.1 Run my_drawing.py

```
cd LeRobot_Python_EN && python3 my_drawing.py
```

or `make my_drawing`, or drop `my_drawing.py` on `LeRobotPythonEN.app`. A saved drawing looks like this:

```
# My lovely drawing
from LeRobot import *

init_pict(); gc()
fd(); fd(); qtr(); fd(); fd(); pc(2)  # blue
fd(); translate(5, 3); fd(); cl(); fd(); st()
```

3.2 Write your own program

Create a file anywhere, for example `house.py`:

```
from LibRobot import *

# Draw a square
fd(); fd(); qtr(); fd(); fd(); qtr()
fd(); fd(); qtr(); fd(); fd(); qtr()
st()
```

Run it with `./robot.sh path/to/house.py` from the `LeRobot_Python_EN` folder, or drop it on `LeRobotPythonEN.app`.

Every program follows the same pattern:

- `from LibRobot import *` first. It opens the window, puts the robot in the centre, pointing north, pen down, in black, with a grid of 12 points (the settings of the book's programs). `from LeRobot import *` gives the original interface used by `my_drawing.py`: a grid of 30 points, and the program must start with `init_pict(); gc()`.
- The robot starts with its pen down, so every move draws: call `pu()` to move without drawing, and `pd()` to draw again.
- `st()` last: shows the speed screen and keeps the window open. Without it, the bar says *End of the program: press a key or click* and the window closes at the next key or click.
- Commands without arguments take `()`; commands are separated by `;` or written on separate lines.

3.3 Programs of the book

The folder `book` holds the 406 programs of the book *First Programming Lessons in Python*, one subfolder per chapter:

```
./robot.sh book/20_polar_grid/LimaconOfPascal.py
make run PROG=book/21_recursion/DragonCurve
make check                          # all programs, one after the other
make check DIR=02_interaction       # one chapter
make check FROM=FriezeOfTriangles   # resume at a program
```

With `make check`, answer in the robot window when a drawing is finished: Return = correct (next program), `r` = run it again, `q` or Esc = stop (the command to resume is printed), any other key starts a remark, ended by Return, noted in `check.txt`. A program that never ends by itself (`Loop1`, `Loop2`, `DigitalClock`) is stopped with Esc. Programs that ask questions show them in the bottom bar and in the Terminal; answer in either. `LEROBOT_MUET=1` turns the sound off. These programs are the English translations of those of the French edition, and draw exactly the same pictures; `tools/renames` gives the French name of each one.

3.4 Replaying at another speed

When the program reaches `st()`, the bar shows:

```
+: faster   -: slower   speed: N     other key: redraw   Esc: quit
```

- `+` / `-` — change the speed from 0 to 10 (default 10, the fastest). Only the number changes.
- Any other key, for example the space bar — clear the window and redraw at the chosen speed; the speed screen comes back when it is done.

- Esc — close the window.

At speed 1 each command takes about 0.1 s. Speed 0 is step-by-step mode: the robot does one command each time you press a key. While the robot draws, Esc (or `§` or `#`) stops the program.

3.5 Command reference

One step is one grid square. The underlined letters give the mnemonic. Commands with two numbers take them as two arguments: `moveTo(x, y)`.

Command	Meaning
`fd()`	Move forward one step
`fdn(n)`	Move forward n steps (backwards if $n < 0$)
`cr()` / `cl()`	Curve right / left (draws a quarter circle)
`etr()` / `etl()`	Eighth turn right / left (45°)
`qtr()` / `qtl()`	Quarter turn right / left (90°)
`tetr()` / `tetl()`	Three eighths of a turn right / left (135°)
`ht()`	Half turn (180°)
`fn() fne() fe() fse() fs() fsw() fw() fnw()`	Face north, NE, E, SE, S, SW, W, NW
`face(n)`	Face direction n (`north` = 0, `northEast` = 1 ... `northWest` = 7)
`rot(a)`	Rotate by a degrees (anticlockwise), keeping the step length
`pd()` / `pu()`	Pen down / up
`dp()` / `dc()`	Draw a point (a square of side `pw`) / a small cross
`pc(n)`	Pen colour n (`white` = 0, `yellow`, `blue`, `violet`, `green`, `red`, `indigo`, `black` = 7)
`pw(n)`	Pen width n (a square pen, as on the Macintosh)
`bg(n)`	Clear the window and fill it with background colour n
`paint()`	Paint the closed area behind the robot with the pen colour
`translate(dx, dy)`	Move dx steps across and dy steps up, without drawing
`drawTo(x, y)`	Go to window pixel (x, y), drawing if the pen is down
`moveTo(x, y)`	Go to grid point (x, y), counted from the centre
`fdl(l)`	Move forward a length of l points, the grid following the robot
`gc()`	Go to the centre
`gt() gb() gr() gl()`	Go to the top, bottom, right or left edge
`dg()`	Draw the grid points
`gs(x)` / `gsX(x)` / `gsY(x)`	Set the grid size (both directions / across / up)
`gsRT(r, t)`	Grid step of length r at angle t (polar grid)
`showRobot()` / `hideRobot()`	Show / hide the robot
`allowExits()` / `forbidExits()`	Allow / forbid moving past the window edge
`sp(n)`	Set the speed (0 to 10)
`ss()` / `ex()`	Step by step / normal execution (full speed)
`message("text")`	Write a message in the bottom bar
`label("text")`	Write a text at the robot's position

Command	Meaning
`readInt()` / `readChar()`	Read an integer / a key
`write(...)` / `writeln(...)`	Write in the Terminal
`st()`	End (stop): show the speed screen

Test functions such as `tb()` (test border: would the next step leave the window?), `tn()` ... `tnw()` (test whether the robot faces north ...) and the values `valX()`, `valGsX()`, `valPc()`, `valOrientation()`, ... are also available; so are the music commands (`note(C, quarter)`, `rest(half)`, `metronome(n)`, `dynamics(n)`), the texts (`textFont`, `textSize`, `intToString`, ...) and the time (`clock()`, `date()`, `delay(t)`). The complete list, with the French name of each command, is in the `README.md` file of `LeRobot_Python_EN` (table `NOMS` of `noms.py`).

4 Troubleshooting

Symptom	Cause	Fix
The program hangs, no window	No Python with Tk 8.6 on this Mac	Install Python from python.org (installation manual)
No module named 'LibRobot'	The program is not in `LeRobot_Python_EN` and was started with `python3`	Use `./robot.sh Program.py` or `LeRobotPythonEN.app`
NameError: name 'av' is not defined	A program written for the French edition	Run it with `LeRobot_Python_FR`, or replace the French names (`tools/fr2en.py` does it)
Error line N of Prog.py: the robot would leave the frame (fd)	The robot was asked to move past the window edge	Change the program, or call `allowExits()`
`** EXIT FORBIDDEN ** translate` in the terminal	In Draw, a move past the window edge	Only a warning
Error line N of Prog.py: endless recursion (stack full)	A procedure that calls itself forever	Check the stopping case of the recursion
The program never ends	An endless loop (`while True`)	Press Esc in the robot window
Keys do nothing in the window	The window does not have keyboard focus	Click in the window
A saved drawing was lost	Saving from Draw always overwrites `my_drawing.py`	Copy `my_drawing.py` under a new name before saving again
Text font cannot be changed	`textFont` and `textStyle` have no effect	None; texts use Helvetica 13